

Human-Centric Artificial Intelligence Project-3 Report

Name – Surname: Barış Kaplan

Matriculation Number: 669584

In this project, the following class mappings are used:

Class 0 => "World"

Class 1 => "Sports"

Class 2 => "Business"

Class 3 => "Sci/Tech"

Task 1:

Used Classification Model: Logistic Regression

Classifier Baseline Test Accuracy Achieved: 90.55%

```
# Split the imported data into train and test parts
self.X_train_raw = dataset['train']['text']
self.y_train = np.array(dataset['train']['label'])
self.X_test_raw = dataset['test']['text']
self.y_test = np.array(dataset['test']['label'])

# Text Feature Extraction
self.vectorizer = TfidfVectorizer(max_features=5000, stop_words='english')
self.X_train = self.vectorizer.fit_transform(self.X_train_raw).toarray()
self.X_test = self.vectorizer.transform(self.X_test_raw).toarray()

# Baseline Classifier
self.baseline_model = LogisticRegression(C=0.4, max_iter=200)
self.baseline_model.fit(self.X_train, self.y_train)
self.baseline_test_preds = self.baseline_model.predict(self.X_test)
self.baseline_acc = float(accuracy_score(self.y_test, self.baseline_test_preds))
```

Code used for train test splitting, text feature extraction, and baseline model training, and baseline model evaluation using accuracy score between the ground truth y_test data and baseline model test data predictions

Task 2:

Simulated Expert's Test Dataset Accuracy Achieved: 92.70% (Specialized expertise in Class 1 (Sports))

```
# Simulated Expert
self.expert_test_preds = self.simulate_expert_predict(self.y_test)
self.expert_acc = float(accuracy_score(self.y_test, self.expert_test_preds))

def simulate_expert_predict(self, y_true):
    """ Simulates bounded human expert decisions. """
    expert_preds = []
    for label in y_true:
        # Specific domain specialization
```

```

if label == 1:
    expert_preds.append(label)
else:
    if np.random.rand() < 0.90:
        expert_preds.append(label)
    else:
        remaining_classes = [c for c in range(4) if c != label]
        expert_preds.append(np.random.choice(remaining_classes))
return np.array(expert_preds)

```

Code implemented for the predictions of a simulated expert and evaluation of the expert using the accuracy score between the ground-truth y_{test} data and the expert predictions on the y_{test} data

```

def simulate_expert_predict(self, y_true):
    """Simulates bounded human expert decisions."""
    expert_preds = []
    for label in y_true:
        # Specific domain specialization
        if label == 1:
            expert_preds.append(label)
        else:
            if np.random.rand() < 0.90:
                expert_preds.append(label)
            else:
                remaining_classes = [c for c in range(4) if c != label]
                expert_preds.append(np.random.choice(remaining_classes))
    return np.array(expert_preds)

```

Code implemented for the predictions of the simulated expert

Strengths of the Simulated Expert on the Dataset

1. Domain Expertise in Specific Regions of the Input Space: This simulated expert exhibits a deterministic behavior for Class 1, which is the “Sports” field. It suitably mimics the knowledge of real-world human experts who possess deep knowledge in a specific field (Class 1: Sports), instead of holding a generalized and/or broad expertise. Therefore, it will serve as a realistic and resource-efficient proxy substitute to evaluate cost-efficient (efficient in terms of time, computational complexity, and financial budget) routing and decision delegation mechanisms within the learning-to-defer framework.

2. High Accuracy Achieved on the Test Dataset: The simulated expert has achieved a high test dataset accuracy of “92.70%”, which means it is a highly reliable ground-truth substitute.

3. Significant Reduction of Time, Effort, and Cost Needed for Manual Data Labeling: This simulated expert can serve as a highly reliable proxy for a manual domain expert workforce on the ground-truth dataset; as a result, this approach will significantly reduce the time, effort, and cost required for manual data labeling.

Weaknesses of the Simulated Expert on the Dataset

1. Uniform Error Distribution: The simulated expert makes mistakes %10 of his/her choices. A class for the given text is randomly and uniformly chosen from the remaining options, with equal probability for each class option. However, this is not the case for the decisions of real-life human experts. Human experts rarely guess uniformly and randomly. They are typically influenced by cognitive biases, systematic biases, patterned behavior, and social context, which makes them usually confuse specific classes sharing similar features.

```
else:
    if np.random.rand() < 0.90:
        expert_preds.append(label)
    else:
        remaining_classes = [c for c in range(4) if c != label]
        expert_preds.append(np.random.choice(remaining_classes))
```

2. Class Imbalance Problem: When human experts encounter an unclear text, they will inherently develop a high level of prior bias towards the classes with more dominance than the others. However, my simulated expert cannot observe the frequencies of the classes (labels). Therefore, its guessing behavior and following prediction errors do not change based on which classes are seen more often, which means the simulated expert is not affected by the class distribution.

3. Static Error Ratio: The error ratio is static and 10% for all remaining classes except Class 1, which are Class 0, 2, and 3. However, the human experts' data labeling performance and errors made in this process change a lot based on several factors, such as tiredness level, daytime, and changing expertise levels for the remaining classes.

4. No Contextual Knowledge of the Input X Values: Even though the simulated expert knows the ground-truth class labels (Class 0 => "World", Class 1 => "Sports", Class 2 => "Business", Class 3 => "Sci/Tech"), it does not know how the features of the input values differ from each other in terms of the length, number, meaning, complexity, and structure of the words in each input text.

5. No "I am completely unsure" Option: This expert is forced to select from the available labels in the error case. However, our expert in real-life might not be able to classify the input into the provided categories and put it into a category like "Unknown".

Task 3:

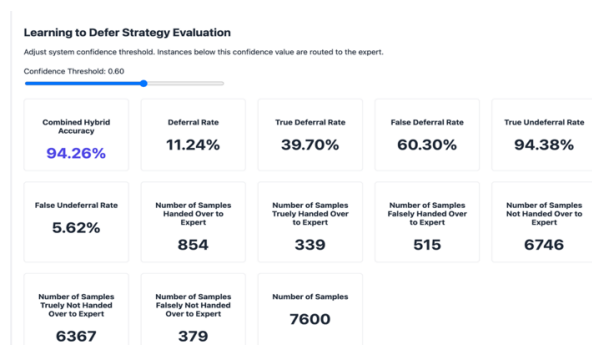


Figure 1: Learning to Defer Strategy Evaluation When Confidence Threshold is 0.60

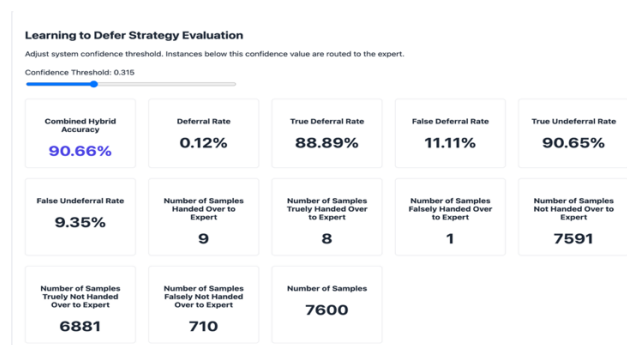


Figure 2: Learning to Defer Strategy Evaluation When Confidence Threshold is 0.315



Figure 3: Learning to Defer Strategy Evaluation When Confidence Threshold is 0.835

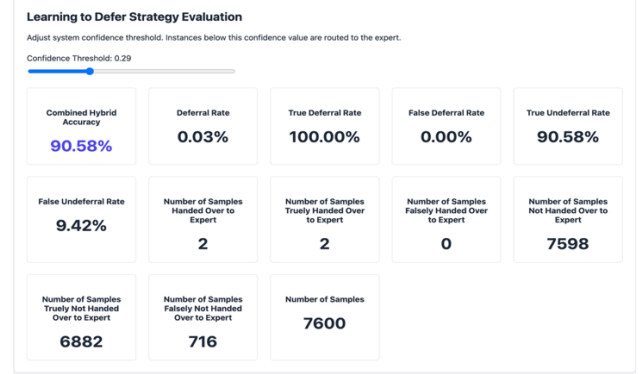


Figure 4: Learning to Defer Strategy Evaluation When Confidence Threshold is 0.29

```

70 def process_learning_to_defer(self, threshold):
71     """Dispatches low-confidence choices to human or expert pipelines."""
72     probs = self.baseline_model.predict_proba(self.X_test)
73     max_probs = np.max(probs, axis=1)
74     classifier_test_preds = np.argmax(probs, axis=1)
75
76     final_predictions = []
77
78     deferral_count = 0
79     undeferred_count = 0
80
81     true_deferral_count = 0
82     false_deferral_count = 0
83
84     true_undeferred_count = 0
85     false_undeferred_count = 0
86
87
88     for i in range(len(self.X_test)):
89         is_classifier_correct = (classifier_test_preds[i] == self.y_test[i])
90
91         if max_probs[i] < threshold:
92             final_predictions.append(self.expert_test_preds[i])
93             deferral_count += 1
94
95             if not is_classifier_correct:
96                 true_deferral_count += 1
97             else:
98                 false_deferral_count += 1
99
100         else:
101             final_predictions.append(classifier_test_preds[i])
102             undeferred_count += 1
103
104             if is_classifier_correct:
105                 true_undeferred_count += 1
106             else:
107                 false_undeferred_count += 1
108
109     system_accuracy = float(accuracy_score(self.y_test, final_predictions))
110     denom_deferral = true_deferral_count + false_deferral_count
111     true_deferral_rate = float(true_deferral_count / denom_deferral) if denom_deferral > 0 else 0.0
112     false_deferral_rate = float(false_deferral_count / denom_deferral) if denom_deferral > 0 else 0.0
113
114     denom_undeferred = true_undeferred_count + false_undeferred_count
115     true_undeferred_rate = float(true_undeferred_count / denom_undeferred) if denom_undeferred > 0 else 0.0
116     false_undeferred_rate = float(false_undeferred_count / denom_undeferred) if denom_undeferred > 0 else 0.0
117
118     deferral_rate = float(deferral_count / len(self.X_test)) if len(self.X_test) > 0 else 0.0
119
120     return {
121         'system_accuracy': system_accuracy,
122         'deferral_rate': float(deferral_count / len(self.X_test)),
123         'true_deferral_rate': true_deferral_rate,
124         'false_deferral_rate': false_deferral_rate,
125         'true_undeferred_rate': true_undeferred_rate,
126         'false_undeferred_rate': false_undeferred_rate,
127         'deferral_count': deferral_count,
128         'undeferred_count': undeferred_count,
129         'true_deferral_count': true_deferral_count,
130         'false_deferral_count': false_deferral_count,
131         'true_undeferred_count': true_undeferred_count,
132         'false_undeferred_count': false_undeferred_count,
133         'total_count': len(self.X_test)
134     }

```

```

95     if not is_classifier_correct:
96         true_deferral_count += 1
97     else:
98         false_deferral_count += 1
99
100     else:
101         final_predictions.append(classifier_test_preds[i])
102         undeferred_count += 1
103
104         if is_classifier_correct:
105             true_undeferred_count += 1
106         else:
107             false_undeferred_count += 1
108
109     system_accuracy = float(accuracy_score(self.y_test, final_predictions))
110     denom_deferral = true_deferral_count + false_deferral_count
111     true_deferral_rate = float(true_deferral_count / denom_deferral) if denom_deferral > 0 else 0.0
112     false_deferral_rate = float(false_deferral_count / denom_deferral) if denom_deferral > 0 else 0.0
113
114     denom_undeferred = true_undeferred_count + false_undeferred_count
115     true_undeferred_rate = float(true_undeferred_count / denom_undeferred) if denom_undeferred > 0 else 0.0
116     false_undeferred_rate = float(false_undeferred_count / denom_undeferred) if denom_undeferred > 0 else 0.0
117
118     deferral_rate = float(deferral_count / len(self.X_test)) if len(self.X_test) > 0 else 0.0
119
120     return {
121         'system_accuracy': system_accuracy,
122         'deferral_rate': deferral_rate,
123         'true_deferral_rate': true_deferral_rate,
124         'false_deferral_rate': false_deferral_rate,
125         'true_undeferred_rate': true_undeferred_rate,
126         'false_undeferred_rate': false_undeferred_rate,
127         'deferral_count': deferral_count,
128         'undeferred_count': undeferred_count,
129         'true_deferral_count': true_deferral_count,
130         'false_deferral_count': false_deferral_count,
131         'true_undeferred_count': true_undeferred_count,
132         'false_undeferred_count': false_undeferred_count,
133         'total_count': len(self.X_test)
134     }

```

Code implemented for the learning-to-defer process

Here, the system accuracy calculated is a hybrid value. The accuracy value is calculated for the ground truth "y_test" values and final_predictions, which include both the expert predictions and baseline classifier predictions. To avoid a divide-by-zero error in the ratio

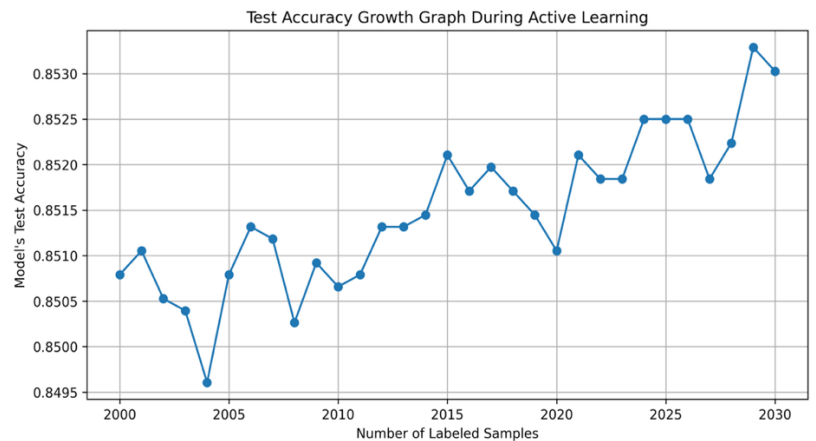
values, we checked whether the denominator of each ratio value is greater than 0; if so, it is used as it is. Otherwise, the corresponding ratio value is taken as 0.0.

If the max probability coming from the baseline model trained on the X input test dataset is below the threshold (**max_probs[i] < threshold**), it means that we are deferring to the expert. However, in this scenario, if the baseline classifier is correct for the current index, it is a **false deferral (false positive)** of knowledge to the expert. Otherwise, it is a **true deferral (true positive)** case.

If the max probability coming from the baseline model trained on the X input test dataset is above the threshold (**max_probs[i] >= threshold**), it means that we do not defer to the expert, meaning the baseline model is confident enough in its prediction. However, in this scenario, if the baseline classifier is correct for the current index, it is a **true undeferral (true negative)** of knowledge to the expert. Otherwise, it is a **false undeferral (false negative)** case.

Task 4 & Task 5:

```
project3 > active_learning_evaluation.py >
1 from ml_backbone import ml_manager
2
3
4 initial_disagreement = ml_manager.get_disagreement_metrics()
5 print("Baseline Model Test Accuracy: (ml_manager.baseline_acc:.4f)")
6 print("Simulated Expert Test Accuracy: (ml_manager.expert_acc:.4f)")
7 print("Initial Deferral Agreement Rate: (initial_disagreement['agreement_rate']:.4f)")
8 print("Initial Deferral Disagreement Rate: (initial_disagreement['disagreement_rate']:.4f)\n")
9
10 # Active Learning Simulation Loop
11 print("Starting Active Learning Queries...")
12 num_queries_to_run = 30
13 print("Number of active learning queries to run: (num_queries_to_run)")
14
15 initial_ml_accuracy = ml_manager.accuracy_history(0)
16 print("The active learning model's test accuracy at Query 0 (Initialization) is: (initial_ml_accuracy:.4f)\n")
17
18 for i in range(num_queries_to_run):
19     # Get the next most uncertain sample
20     sample_data = ml_manager.get_next_uncertain_sample()
21     if sample_data is None:
22         break
23     # Add the expert's label into the update system
24     update_result = ml_manager.process_query_update(
25         sample_data[sample_index],
26         chosen_label=sample_data[simulated_expert_label]
27     )
28     # Log the progress
29     print(f"Query ({i+1}/{num_queries_to_run}) | "
30           f"Test Accuracy: (update_result['current_test_accuracy']:.4f) | "
31           f"Accuracy Gain/Loss Per Query: (update_result['accuracy_gain_loss_per_query']:.4f) | "
32           f"Remaining Deferral Ops: (update_result['total_deferral_opportunities'])")
33
34 # Final Visualization of the Active Learning Process
35 print("\nPlotting results...")
36 ml_manager.plot_metrics()
```



Code implemented for the active learning evaluation and test accuracy change graph plotting during active learning process

Figure 5: Number of Labeled Samples vs Model's Test Accuracy Graph for 30 Active Learning Queries

```
(hcal_env) (base) bariskaplan@Baris-MacBook-Pro HCAI-PBL-Projects % ls
~$project3_Report.docx      hcal_env      manage.py      pdf_files_of_projects      project3
project4                    templates     media          project1        Project3_Report.docx
db-solite3                  iris.csv      pbl            project2        Project3_Report.pdf
demo                        static
(hcal_env) (base) bariskaplan@Baris-MacBook-Pro HCAI-PBL-Projects % cd project3
(hcal_env) (base) bariskaplan@Baris-MacBook-Pro project3 % ls
__init__.py      active_learning_evaluation.py  migrations      README.md
__pycache__      iris.py                    admin.py        ml_backbone.py  requirements.txt
views.py          accuracy_growth_graph.png    apps.py        models.py        tests.py
(hcal_env) (base) bariskaplan@Baris-MacBook-Pro project3 % python active_learning_evaluation.py
Number of news in the training dataset: 120000
Number of news in the testing dataset: 7000
Baseline Model Test Accuracy: 0.9052
Simulated Expert Test Accuracy: 0.9278
Initial Deferral Agreement Rate: 0.8626
Initial Deferral Disagreement Rate: 0.1374

Starting Active Learning Queries ...
Number of active learning queries to run: 30
The active learning model's test accuracy at Query 0 (Initialization) is: 0.8508

Query 1/30 | Test Accuracy: 0.8511 | Accuracy Gain/Loss Per Query: 0.0003 | Remaining Deferral Ops: 1042
Query 2/30 | Test Accuracy: 0.8505 | Accuracy Gain/Loss Per Query: -0.0005 | Remaining Deferral Ops: 1046
Query 3/30 | Test Accuracy: 0.8504 | Accuracy Gain/Loss Per Query: -0.0001 | Remaining Deferral Ops: 1046
Query 4/30 | Test Accuracy: 0.8496 | Accuracy Gain/Loss Per Query: -0.0008 | Remaining Deferral Ops: 1052
Query 5/30 | Test Accuracy: 0.8508 | Accuracy Gain/Loss Per Query: 0.0012 | Remaining Deferral Ops: 1041
Query 6/30 | Test Accuracy: 0.8513 | Accuracy Gain/Loss Per Query: 0.0005 | Remaining Deferral Ops: 1041
Query 7/30 | Test Accuracy: 0.8512 | Accuracy Gain/Loss Per Query: -0.0001 | Remaining Deferral Ops: 1041
Query 8/30 | Test Accuracy: 0.8503 | Accuracy Gain/Loss Per Query: -0.0009 | Remaining Deferral Ops: 1047
Query 9/30 | Test Accuracy: 0.8509 | Accuracy Gain/Loss Per Query: 0.0007 | Remaining Deferral Ops: 1044
Query 10/30 | Test Accuracy: 0.8507 | Accuracy Gain/Loss Per Query: -0.0003 | Remaining Deferral Ops: 1046
Query 11/30 | Test Accuracy: 0.8508 | Accuracy Gain/Loss Per Query: 0.0001 | Remaining Deferral Ops: 1045
Query 12/30 | Test Accuracy: 0.8513 | Accuracy Gain/Loss Per Query: 0.0005 | Remaining Deferral Ops: 1041
Query 13/30 | Test Accuracy: 0.8513 | Accuracy Gain/Loss Per Query: 0.0000 | Remaining Deferral Ops: 1041
Query 14/30 | Test Accuracy: 0.8514 | Accuracy Gain/Loss Per Query: 0.0001 | Remaining Deferral Ops: 1040
Query 15/30 | Test Accuracy: 0.8521 | Accuracy Gain/Loss Per Query: 0.0007 | Remaining Deferral Ops: 1035
Query 16/30 | Test Accuracy: 0.8517 | Accuracy Gain/Loss Per Query: -0.0004 | Remaining Deferral Ops: 1038
Query 17/30 | Test Accuracy: 0.8520 | Accuracy Gain/Loss Per Query: 0.0003 | Remaining Deferral Ops: 1036
Query 18/30 | Test Accuracy: 0.8517 | Accuracy Gain/Loss Per Query: -0.0003 | Remaining Deferral Ops: 1038
Query 19/30 | Test Accuracy: 0.8514 | Accuracy Gain/Loss Per Query: -0.0003 | Remaining Deferral Ops: 1040
Query 20/30 | Test Accuracy: 0.8511 | Accuracy Gain/Loss Per Query: -0.0004 | Remaining Deferral Ops: 1040
Query 21/30 | Test Accuracy: 0.8521 | Accuracy Gain/Loss Per Query: 0.0011 | Remaining Deferral Ops: 1036
Query 22/30 | Test Accuracy: 0.8518 | Accuracy Gain/Loss Per Query: -0.0003 | Remaining Deferral Ops: 1038
Query 23/30 | Test Accuracy: 0.8518 | Accuracy Gain/Loss Per Query: 0.0000 | Remaining Deferral Ops: 1038
Query 24/30 | Test Accuracy: 0.8525 | Accuracy Gain/Loss Per Query: 0.0007 | Remaining Deferral Ops: 1033
Query 25/30 | Test Accuracy: 0.8525 | Accuracy Gain/Loss Per Query: 0.0000 | Remaining Deferral Ops: 1034
Query 26/30 | Test Accuracy: 0.8525 | Accuracy Gain/Loss Per Query: 0.0000 | Remaining Deferral Ops: 1034
Query 27/30 | Test Accuracy: 0.8518 | Accuracy Gain/Loss Per Query: -0.0007 | Remaining Deferral Ops: 1037
Query 28/30 | Test Accuracy: 0.8522 | Accuracy Gain/Loss Per Query: 0.0004 | Remaining Deferral Ops: 1036
Query 29/30 | Test Accuracy: 0.8523 | Accuracy Gain/Loss Per Query: 0.0011 | Remaining Deferral Ops: 1030
Query 30/30 | Test Accuracy: 0.8530 | Accuracy Gain/Loss Per Query: -0.0003 | Remaining Deferral Ops: 1031

Plotting results...
Plot successfully saved to: accuracy_growth_graph.png
(hcal_env) (base) bariskaplan@Baris-MacBook-Pro project3 %
```

Figure 6: Active Learning Evaluation Results and Progress for 30 Active Learning Queries

```

140 def get_next_uncertain_sample(self):
141     """Selects data instances with lowest prediction confidence scores."""
142     if not self.AL_pool_indices:
143         return None
144
145     pool_probs = self.active_learning_model.predict_proba(self.X_train[self.AL_pool_indices])
146     max_pool_probs = np.max(pool_probs, axis=1)
147
148     uncertain_pool_idx = np.argmax(max_pool_probs)
149     global_idx = self.AL_pool_indices[uncertain_pool_idx]
150
151     simulated_label = self.simulate_expert_predict([self.y_train[global_idx]])[0]
152
153     return {
154         'sample_index': int(global_idx),
155         'text': self.X_train_row[global_idx],
156         'true_label': int(self.y_train[global_idx]),
157         'simulated_expert_label': int(simulated_label),
158         'categories': self.categories
159     }
160
161 def process_query_update(self, idx, chosen_label):
162     """Absorbs feedback loop entries and updates the active learner."""
163     if idx in self.AL_pool_indices:
164         self.AL_pool_indices.remove(idx)
165     if idx not in self.AL_labeled_indices:
166         self.AL_labeled_indices.append(idx)
167
168     self.y_train[idx] = chosen_label
169
170     # Retrain dynamic tracker model instance
171     X_train = self.X_train[self.AL_labeled_indices]
172     y_train = self.y_train[self.AL_labeled_indices]
173     self.active_learning_model.fit(X_train, y_train)
174
175     al_preds = self.active_learning_model.predict(self.X_test)
176     current_al_acc = float(accuracy_score(self.y_test, al_preds))
177     self.accuracy_history.append(current_al_acc)
178     self.query_history.append(len(self.AL_labeled_indices))
179
180     # Get the new disagreement metrics after retraining
181     disagreement_data = self.get_disagreement_metrics()
182
183     return {
184         'status': 'success',
185         'current_test_accuracy': current_al_acc,
186         'total_labeled_count': len(self.AL_labeled_indices),
187         'disagreement_rate': disagreement_data['disagreement_rate'],
188         'disagreement_rate': disagreement_data['disagreement_rate']
189     }

```

```

161 def process_query_update(self, idx, chosen_label):
162     self.accuracy_history.append(current_al_acc)
163     self.query_history.append(len(self.AL_labeled_indices))
164
165     # Get the new disagreement metrics after retraining
166     disagreement_data = self.get_disagreement_metrics()
167
168     return {
169         'status': 'success',
170         'current_test_accuracy': current_al_acc,
171         'total_labeled_count': len(self.AL_labeled_indices),
172         'disagreement_rate': disagreement_data['disagreement_rate'],
173         'total_deferral_opportunities': disagreement_data['total_deferral_opportunities'],
174         'accuracy_gain_per_query': (current_al_acc - self.accuracy_history[-2]) if len(self.accuracy_history) > 1 else 0
175     }
176
177 def plot_metrics(self, save_path="accuracy_growth_graph.png"):
178     plt.figure(figsize=(10, 5))
179     plt.plot(self.query_history, self.accuracy_history, markers='o')
180     plt.title("Test Accuracy Growth Graph During Active Learning")
181     plt.xlabel("Number of Labeled Samples")
182     plt.ylabel("Model's Test Accuracy")
183     plt.grid(True)
184
185     # Save the plot first
186     plt.savefig(save_path, dpi=300, bbox_inches='tight')
187     print(f"Plot successfully saved to: {save_path}")
188
189     # Then, display the plot
190     plt.show()
191
192 def get_disagreement_metrics(self):
193     """Measures how often the active learning model is wrong while the simulated expert is right."""
194     model_preds = self.active_learning_model.predict(self.X_test)
195     model_wrong = (model_preds != self.y_test)
196     expert_right = (self.expert_test_preds == self.y_test)
197
198     # The model is wrong and the expert is right
199     disagreement_mask = model_wrong & expert_right
200     disagreement_rate = np.mean(disagreement_mask)
201     agreement_rate = 1 - disagreement_rate
202
203     return {
204         'agreement_rate': float(agreement_rate),
205         'disagreement_rate': float(disagreement_rate),
206         'total_deferral_opportunities': int(np.sum(disagreement_mask)),
207         'initial_model_accuracy': float(self.accuracy_history[0])
208     }
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225

```

Code implemented for the active learning process and evaluation

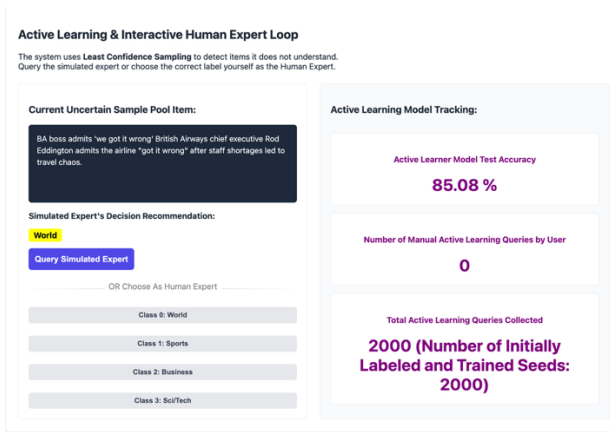


Figure 7: Initial Active Learning Interface State

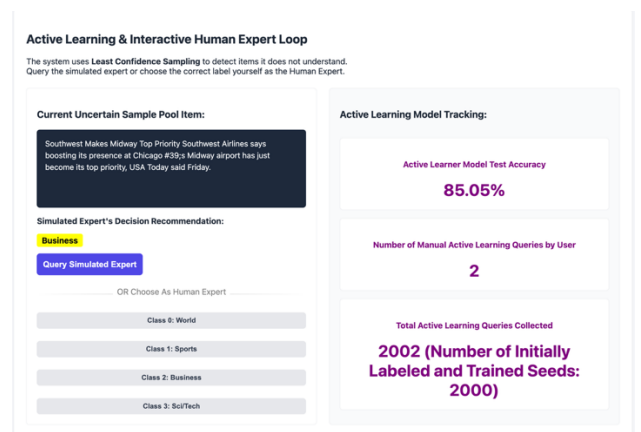


Figure 8: Active Learning Interface State After 2 User Queries

In the active learning process, it is aimed to actively choose which text instance(s) should be presented/deferred to the expert in order to efficiently learn the expert's competence area, the text classification task, and the case where the deferral of knowledge is beneficial, given no access to any expert label during training. In order to achieve this, I have used a sampling strategy named "**Uncertainty-Based (Least-Confidence) Sampling**" in the "**get_next_uncertain_sample()**" function. I have predicted the probability distribution for the unlabeled part of the X training data corresponding to the active learning pool indices. Then, I have selected the maximum value in each row of this probability distribution (using the "**np.argmax(pool_probs, axis=1)**" call) and created a maximum pool probabilities array. At this step, the benefit of deferral needs to be somehow increased. I have achieved this by selecting the global index where the active learning model is the least confident. Here, the least confident index from the pool is obtained by selecting the minimum index value (using the "**np.argmin(values)**" call) from the maximum pool probabilities array created. This suggests that the deferral is more useful because of the lower model confidence at that index.

In the "**process_query_update**" function, I am updating the AL_pool_indices (**pool indices for active learning**) and AL_labeled_indices (**labeled indices for active learning**) lists after labeling each text instance. When I label a text sample, I remove its index from the active learning pool indices and uniquely add it to the labeled indices. Then, I retrain the active learning model with the new list of labeled indices for active learning. After that, I am evaluating the retrained model to get the recent accuracy value of the active learning model trained with new query.

When we look at the test accuracy growth graph for 30 new queries, **given in Figure 5**, we can observe that the overall trend of test accuracy is upward as new queries are provided. However, when an active learner queries highly borderline samples, the decision boundary of the active learning classifier model aggressively changes, which often leads to performance stagnation or decreases on the test set before the self-correction and increase in accuracy.

I have kept track of the current test accuracies of the active learner in *self.accuracy_history* and the number of labeled samples in *self.query_history*. In addition to the graph plotted, I have used deferral agreement & disagreement rates, current test accuracy of the active learning model, and accuracy gain/loss per user query for evaluating the active learning model. They indicate high initial (**0.8508**) & last (**0.8530**) test accuracies, which means active learning performs well on the unseen test data. In order to calculate the disagreement rate between the active learning model and the simulated expert, I have first calculated the part where **the active learning model predictions on the unseen 'y_test' data do not match the ground truth 'y_test' data**. Then, I have calculated the part where **the simulated expert predictions on the unseen 'y_test' data do match the ground truth 'y_test' data**. After that, I get the "AND" mask of these parts. Finally, I have applied *np.mean()* on the created mask, which treats the true boolean value as 1 and the false boolean value as 0. The *np.mean()* call will then give the deferral disagreement rate, which indicates how often the active learning model is wrong while the simulated expert is right.